Certificate-Gated Neural Proposers for Verifier-Grounded Action Scoring

Anonymous authors

Abstract. When an exact verifier is available, the right architecture for action scoring is not an end-to-end neural policy. The useful pattern is a **verifier-grounded ranking loop**: enumerate candidates, certify or replay their outcomes, let a small neural component rank only the residual, and write accepted rules, rejected candidates, replay rows, and reusable hypotheses back into trace artifacts. We instantiate this loop on modular gate search, grid paths, delayed claims, ARC-AGI transforms, program synthesis, and Signal flight replay. The strongest clean gate result is `prime_gap`: with residue-sieve features removed, the raw scorer reaches 9.13% top-1, a fixed hand-wheel gate reaches 45.12%, and a certified learned-rule gate reaches 68.87% at 100% top-10. The same contract transfers outside arithmetic: grid's cheap-bound gate lifts 75.80% raw to 90.10% and its mixed gate reaches 95.30%, delayed-claim's threshold+margin gate reduces the clean held-out action set from four candidates to one with zero false rejections, and Signal's 2113-parameter WASD ranker improves offline replay top-1 and regret over a no-safety goal-heading baseline. ARC is deliberately scoped as certified coverage, not ARC performance. The fixed-budget modular proposer result is negative for naive neural proposal, which is the point: the system becomes useful when rejected candidates become reason-code training signal, certified hypotheses become library priors, and the claim registry blocks unsupported prose before compilation.

1. Introduction

Exact verifiers are common in domains where action correctness can be checked algorithmically even when discovery cannot: theorem proving, program synthesis, combinatorial search, formal shortest-path problems. The dominant style in modern learned policy work treats the verifier as a reward signal for an end-to-end neural policy. We argue this is the wrong default when the candidate space contains many structurally impossible actions that the verifier can cheaply eliminate.

Our thesis is the **certificate-gating loop**: a learned scorer should consume the **survivors** of a verifier-checked symbolic filter, and the filter itself can be produced by a rule proposer gated by a soundness-certificate verifier. The scorer ranks the residual; the gate eliminates the rest; nothing the gate emits can violate correctness because every rule it accepts carries a deterministic proof.

We make three protocol claims, each of which must be instantiated across tasks before being treated as general:

(C1) **Certified gates reduce the residual action space.** The complete certified modular gate result, the smaller grid-path legality/distance result, and the clean delayed-claim threshold+margin gate support this. Delayed-claim's shifted/noisy rows are now a robustness boundary, not an implementation gap.

(C2) **Rule search must beat fixed-budget symbolic baselines.** The current modular rule-search result is negative for neural proposer learning: grammar order beats the neural proposer on the unconditional candidate grammar, and the conditional gap grammar makes the failure larger. This turns proposer learning into a concrete cross-task benchmark rather than a tautology.

(C3) **Verifier RL must preserve calibration across tasks.** Strict policy gradient on sparse exact rewards inflates logit scale on the prime-gap modular gate instance; a standard KL-to-prior +

entropy term restores calibration at unchanged top-k. The same run is still needed on grid paths and delayed claims.

Operationally, the certificate-gating loop yields two primary contributions: an artifact-feedback loop that recycles verifier outputs into future gates, priors, and ranking targets (Table 2), and a generalized task contract that keeps the same candidate/verifier/scorer/trace interface across domains from arithmetic to replay-grounded control (Table 3).

The evidence is heterogeneous by design, so the paper uses a single protocol matrix rather than a single leaderboard. The primary question in every row is not "how smart is the small network?", but "what does the environment supply, what does the verifier own, what is the network allowed to rank, and what reusable artifact is produced?"

Domain	Candidates	Verifier / Replay	Neural Role	Reusable Artifact	Supported Claim
`prime_gap`	129 gaps + modular rules	segmented sieve + rule certificates	rank residual gaps and candidate rules	accepted/rejected rule traces	68.87% learned gate top-1, zero false rejections
`grid_path`	four moves + local rules	shortest-path oracle	rank legal residual moves	certified local predicates	cheap-bound gate lifts raw 75.80% to 90.10%
`delayed_claim`	four delayed actions	hidden-outcome simulator	rank actions, then replay successor	parent-hash replay rows	clean gate reduces four actions to one
`ARC`	colors + transform programs	train-pair consistency + held-out grid	rank cells / programs / hypotheses	certified transform evidence	coverage floor, not ARC-performance claim
`prog_synth`	typed DSL abstractions	example execution + type check	rank reusable programs	hypothesis evidence rows	same hypothesis-to-certificate path beyond ARC
`Signal`	nine WASD candidates	counterfactual simulator replay	rank bounded control actions	utility/damage/death replay rows	offline replay-control vignette

Table 1: Protocol evidence matrix. Every experiment instantiates the same contract: environment-supplied candidates, verifier-owned correctness, neural residual ranking, and reusable trace artifacts.

The artifact now emits `crlp.experiment_schema.v1` rows for every certified task with `make experiment-schemas`. The paper should be read against that schema: task-specific verifiers and grammars change, but candidate enumeration, leakage discipline, proposer baselines, trace replay, and verifier-RL comparisons are shared.

2. Related Work

2.1. Verifiable rewards and learned verifiers

DeepSeek-R1 is the closest contemporary confirmation of the verifier-first premise at frontier scale: DeepSeek-AI report reinforcement learning from verifiable rewards (RLVR), where math, code, and format checks provide correctness signals without a learned reward model [1]. That supports the architectural direction of this paper, but it is not the same artifact. DeepSeek-R1 couples verifiable rewards to massive language-model sampling and opaque training infrastructure; this work keeps the verifier line auditable by putting candidate enumeration, certificate checks, accepted rules, rejected proposals, and scorer metrics into schema-attested C11 traces. The contrast with the standard learned-reward line [2], [3], [4] is therefore not "RL is unnecessary"; it is narrower: when an exact verifier exists, learned components should rank or propose within a verifier-bounded search, not replace the correctness oracle.

The learned-verifier line starts from a different point. GSM8K-style verifier-guided decoding trains a model to rank sampled solutions after generation [5]; process-supervision work trains reward models

on intermediate reasoning steps rather than only final outcomes [6], [7]; V-STaR adds a self-training loop in which both correct and incorrect sampled solutions improve the verifier [8]. Those systems are natural relatives because they all use a learned component to reduce search over candidate reasoning traces. The boundary in this paper is that the verifier is not learned: verifier outputs may provide labels, rewards, certificates, and diagnostics, but accepted gates require deterministic certificates and scorer inputs are checked against a typed leakage policy. In short, learned verifiers ask "which sampled answer should we trust?"; this work asks "which candidates remain after a sound filter, and how should we rank that residual?"

The regularization used in Axis 3 is also borrowed, not claimed as novel. RLHF systems made KL-to-prior and entropy-style stabilization standard for preference rewards [3], [4], with PPO and trust-region policy optimization as the broader policy-gradient background [9], [10]. Our contribution is to show the same stabilization pattern under exact verifier rewards: strict policy-gradient updates inflate confidence on the prime-gap modular gate instance, while the KL/entropy branch preserves ranking and improves calibration. Calibration itself is reported in the standard ECE/temperature-scaling frame of [11].

2.2. Neural-symbolic search with formal checkers

AlphaGeometry is the closest structural ancestor of this work. It interleaves a neural language model that proposes auxiliary geometric constructions with a symbolic deduction engine that verifies whether the construction completes the proof [12]; AlphaGeometry2 extends that line to stronger olympiad-geometry performance [13]. GPT-f is the earlier theorem-proving version of the same pattern: a language model proposes proof steps and a formal system checks them [14]. Our architecture is deliberately smaller and more explicit. The grammar is task-local, the proposer sees structural rule descriptors rather than a full natural-language proof context, and every accepted rule is emitted into a trace with support, marginal utility, certificate status, and false-rejection counts. The comparison is therefore not scale; it is audit posture. Their systems show that neural proposal plus formal checking is powerful at large scale; this paper asks how much of that pattern can be made schema-attested, reproducible, and heterogeneous across small verifier-equipped tasks.

DreamCoder and Hypothesis Search are the closest relatives of the hypothesis layer rather than the modular rule gate. DreamCoder learns reusable DSL abstractions through wake-sleep Bayesian program learning [15]; Hypothesis Search samples natural-language hypotheses, compiles them into executable programs, and keeps the train-consistent ones [16]. `NNHypothesis` sits between those two poles: templates and slot types are deterministic, but bindings are induced from task evidence and compiled into canonical `NNProgram` rows that the same certificate/proposer machinery can score. This is why the ARC and `program_synthesis` evidence rows matter: they show the substrate's "hypothesis -> program -> certificate -> proposer summary" path outside the current modular arithmetic grammar.

2.3. ARC and abstraction benchmarks

ARC is used here as one hard abstraction substrate in the evidence portfolio. Chollet's ARC framing emphasizes skill-acquisition efficiency and the need to account for priors [17]. The 2024 ARC Prize report and recent test-time-training work show the current leaderboard direction: large-model or program-synthesis systems with per-instance adaptation achieve far higher ARC scores than this artifact [18], [19]. Our public ARC result should therefore be read narrowly: grammar v10 certifies a small public-corpus coverage floor and the fixed `0520fde7` transform gate gives a clean scorer-vs-certificate contrast on one task. The research claim is not ARC performance; it is that feature provenance, certificate-gated transforms, matched proposer budgets, and replayable evidence rows make the priors and failure modes visible.

3. Method

3.1. The Certificate-Gating Loop

The reusable architecture is:

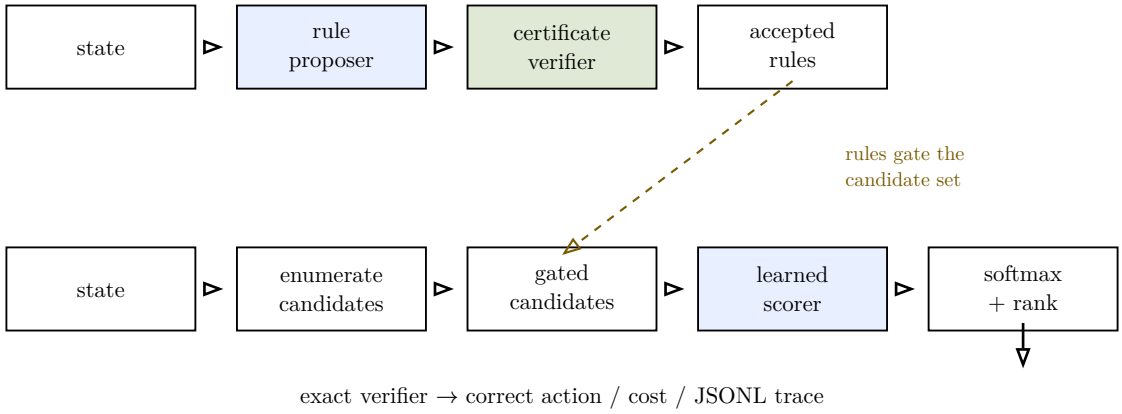


Figure 1: The certificate-gating loop. A rule proposer (top) produces candidate rules that a soundness-certificate verifier accepts or rejects. Accepted rules deterministically prune the candidate set; a shared neural scorer ranks the survivors; the exact verifier supplies the correctness signal.

The design invariant is **the model proposes, the verifier decides, the trace remembers.** A rule r is accepted only if a deterministic algebraic check certifies that r cannot reject the true action on any state in the support set, so soundness does not depend on the proposer being well-trained.

Verifier Output	Trace Artifact	Reusable Form	Next Ranking Use
accepted rule	rule trace with support, certificate status, and false rejections	gate program	prune impossible candidates before scoring
rejected candidate	reason-coded verifier rejection row	unsafe / no-support / duplicate prior	avoid low-utility proposals at fixed budget
certified hypothesis	typed program evidence row	library prior over template slots	promote reusable transforms under a verifier budget
replay outcome	counterfactual utility, damage, and death row	bounded action utility target	rank control candidates without owning safety

Table 2: Artifact feedback loop. The paper's central mechanism is not raw neural accuracy. The environment produces certified or replayed artifacts; those artifacts become gates, priors, and ranking targets for the next proposal pass.

3.2. Generalized Task Schema

Every verifier-backed experiment in the artifact emits the same schema row:

```
Task T = {
  state distribution S_T,
  candidate enumerator A_T(s),
  observable features x_T(s, a),
  neural residual scorer f_theta(x_T),
  exact verifier V_T(s, a),
  symbolic grammar G_T,
  certificate checker C_T(rule),
  rule proposer q_phi(rule | structural(rule)),
  gate executor R_T(rule, s, a),
```

```
  trace/replay/RL protocol
}
```

Task	Candidate Set	Verifier	Certificate / Gate	Scorer Role	Trace Artifact	Protocol Point
`prime_gap`	candidate gaps + modular rule candidates	segmented sieve next-prime verifier	divisibility certificate; zero false rejection	rank residual gaps and budgeted rules	`crlp.compare.v1`, `crlp.rule_trace.v1`	certified modular gate search anchor
`grid_path`	four moves + legality/distance rules	BFS shortest-path successor	move cannot reject optimal successor	rank legal residual moves	`crlp.compare.v1`	same gate contract outside prime residues
`delayed_claim`	trust / ignore / request / cautious	deterministic hidden-outcome simulator	finite-generator safety check	rank actions under delayed labels	`crlp.trace.v1` replay rows	trace replay plus production threshold+margin gate
`arc_agi`	cell colors + transform/program candidates	held-out grid and train-pair transform check	train consistency + unique compiled prediction	rank cells, programs, and hypothesis-derived transforms	`crlp.arc_transform.summary.v1`, `crlp.hypothesis.evidence.v1`	abstraction task exposes priors and grammar coverage
`program_synthesis`	list/int DSL abstractions	run candidate abstraction on examples	monomorphic DSL type check	rank reusable typed programs	`crlp.hypothesis.evidence.v1`	same hypothesis → program → certificate path beyond ARC
`signal_flight`	nine WASD control candidates	authoritative Signal simulator replay	counterfactual utility + damage/death replay	rank bounded real-time control actions	`signal.flight_counterfactual.v1`, `crlp.signal_flight.summary.v1`	application task for better-for-replay grounded control

Table 3: The task instantiations share the same verifier-gated action-scoring contract. Each row repeats the same question: what is the candidate set, which checker owns correctness, what becomes a certificate, what the learned model is allowed to rank, which trace records the decision, and what the task proves about the protocol.

The leakage rule is shared: verifier outputs may be labels, rewards, certificates, diagnostics, or executed gates, but they are not default scorer inputs. Proposer inputs are structural rule descriptors; support-derived marginal utility labels and false-rejection diagnostics are targets, not features. In the artifact this is a typed contract, not just prose: every scorer feature declares provenance (`state_observation`, `structural_prior`, `verifier_derived`, or `derived_from_gate`), every task declares a leakage policy (`forbidden`, `priors_allowed`, or `legacy`), and schema emission fails if enabled scorer features violate that policy.

3.3. Trace and Checkpoint Contract

Every step writes a JSONL row with schema `crlp.trace.v1`, including a stable per-action identifier, the candidate scores and probabilities, the selected and target indices, the verifier result, and verifier cost. Replay resolves `target_index` → stable `action_id` → regenerated candidate row, so reordering action ids cannot silently change replay semantics. Checkpoints carry SHA-256 model identity, a `parent_model_hash`, a `feature_encoder_version`, and a `feature_set`; comparison reports (`crlp.compare.v1`) carry both hashes, the evaluated `feature_set`, and machine-readable reliability bins.

4. Task Family: Certified Modular Gate Search

The prime-gap task is the current production instantiation of certified modular gate search. The family is defined by the proof shape, not by primes: enumerate arithmetic candidates, propose residue/period/divisibility rules from structural descriptors, accept only rules with deterministic certificates, gate impossible candidates, and rank the residual with the learned scorer. Prime constellations, k-free or

polynomial-value filters, and Fibonacci/Lucas recurrence gates are natural next members because they preserve that modular certificate contract while changing the arithmetic substrate.

The artifact now includes a smoke implementation of those next members in `examples/modular_gate_search.c`. It emits `crlp.modular_gate.summary.v1` rows for `prime_constellation`, `kfree_squarefree`, and `fibonacci_lucas` under matched rule budgets. Each task reports `grammar_order`, `random`, `structural_heuristic`, `learned_structural`, `train_marginal_oracle`, and `validation_marginal_oracle` rows. The learned proposer is a structural-only linear model trained on train-set rule coverage labels, and the smoke requires zero held-out false rejections. These rows defend the family boundary; they do not yet replace the production `prime_gap` learned-gate table.

4.1. Prime-Gap Setup

The state is a prime p; the candidate space is gaps $g \in \{2, 4, ..., 256\}$ plus an overflow action; the verifier returns the true next-prime gap. The default candidate encoding is the minimal feature set $[\log p, \text{norm_gap}, \text{overflow_flag}]$. A full-sieve feature set that adds precomputed divisibility flags for $p + g$ modulo small primes is retained only as a leakage ablation. The scorer is a shared MLP returning one scalar per candidate; softmax over candidates produces the proposal distribution.

The default split is $p < 800,000$ train, $800,000 \le p < 1,000,000$ validation, $1,000,000 \le p \le 3,000,000$ test; all reported numbers use 30,000 held-out test states unless stated otherwise.

4.2. Why this domain

- A fast deterministic verifier at all scales (segmented sieves, wheel filters, block proofs).
- A compact totally-ordered candidate space.
- An exhausted classical literature — we cannot beat a sieve, which sharpens the question to "what does the learned component contribute under perfect feedback?"
- Rich algebraic structure for the rule grammar (residue classes, divisibility certificates).

5. Axis 1: Certified gates reduce the residual action space

The same trained prime-gap scorer inside the certified modular gate instance is evaluated under three gate conditions on the canonical 30,000-sample held-out benchmark. Top-1 results:

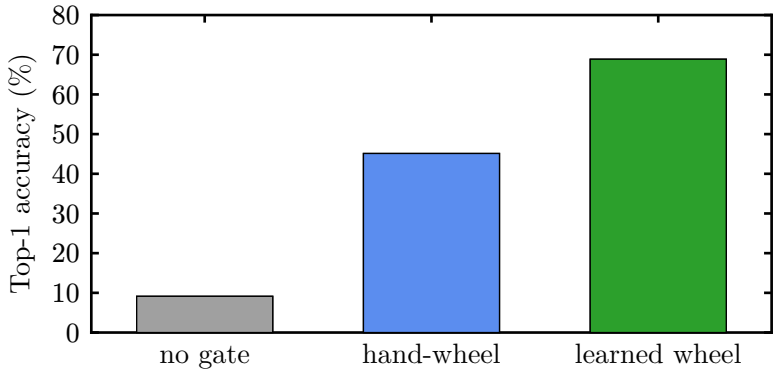


Figure 2: Top-1 accuracy by gate on the canonical 30,000-sample held-out benchmark. The clean raw-to-learned delta is **9.13 → 68.87**. The fixed-hand-wheel-to-learned delta is **45.12 → 68.87** (+23.8pp).

The former 45.12% raw-scorer result is now a full-sieve-feature leakage ablation, not the headline.

Full metrics:

Gate	Top-1	Top-10	NLL	ECE
no gate	9.13%	78.01%	2.9994	0.1076
hand-wheel	45.12%	99.77%	1.5902	0.0834
learned wheel	**68.87%**	**100.00%**	**1.0056**	0.1402

Table 4: Gate ablation, minimal-feature scorer, identical weights across rows. Timing fields are excluded from the canonical comparison because they vary by runner; the numerical claim here is the accuracy and calibration effect of certified pruning.

6. Axis 2: Rule search must beat fixed-budget symbolic baselines

The canonical rule grammar is `reject candidate n when n > m and n % m == r`; the certificate verifier accepts a rule iff $\gcd(m, r) > 1$ (so n is necessarily composite). The matched-budget baseline table also reports conditional gap filters (`if p % m == a then reject gap % m == b`) because that grammar is less saturated and exposes proposer failure more clearly. Zero false rejections is a structural property of the certificate. The proposer input is structural-only: condition kind, moduli, reciprocal scales, residues, zero-residue indicators, gcd ratios, and derived candidate residue. Empirical support and false-rejection counts are diagnostics, and budgeted marginal utility is the neural target, not a proposer feature.

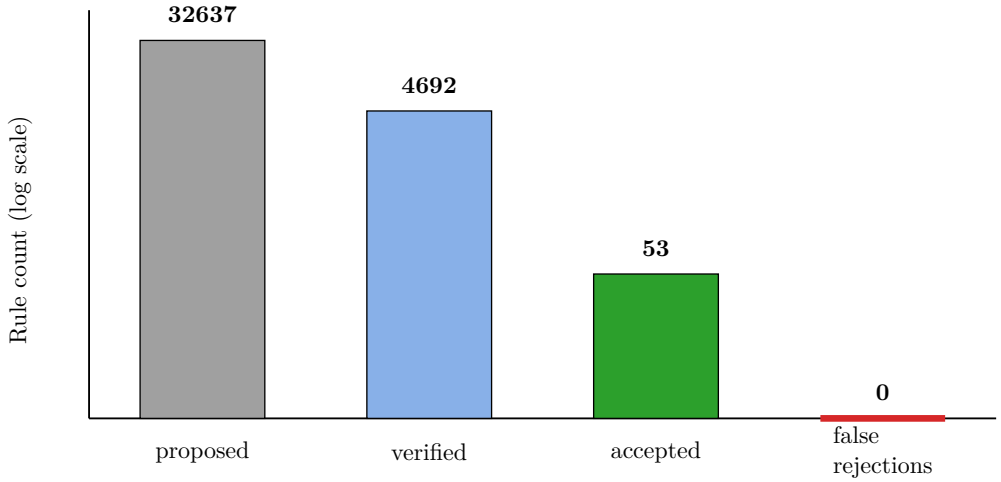


Figure 3: Rule-search acceptance funnel. Of 32,637 proposed rules, 4,692 pass the soundness certificate, 53 survive support and false-rejection screens, and **zero** falsely reject the true next-prime gap. The accepted set recovers 79.79% of wheel-equivalent verifier work.

Grammar	Rule ordering	Budget	Exact probes	Saved
candidate	neural structural	16	1.98	72.49%
candidate	random, seed 20240511	16	3.86	46.30%
candidate	grammar order	16	1.93	73.22%
candidate	structural heuristic	16	1.93	73.22%
candidate	validation-marginal oracle	16	1.93	73.22%
conditional gap	neural structural	16	7.04	2.15%
conditional gap	random, seed 20240511	16	6.94	3.49%
conditional gap	grammar order	16	3.20	55.57%
conditional gap	structural heuristic	16	3.20	55.57%
conditional gap	validation-marginal oracle	16	3.11	56.83%

Table 5: Matched 16-rule-budget proposer baselines over the same split and certificate verifier. Lower exact probes is better.

What this evidence supports. "Certificate-gated structural rule discovery over this grammar produces a sound, useful rule set without support-count feature leakage." The soundness comes from the certificate; the coverage comes from the grammar plus accepted certified filters.

What this evidence does not support. "The neural proposer, specifically, is essential." Table 5 is a negative result for that stronger claim: simple deterministic orderings beat the neural structural proposer at the fixed 16-rule budget on both reported grammars, and the conditional gap grammar exposes a larger failure. The next neurosymbolic experiment must use a richer grammar and show that learned ordering beats grammar-order and structural-heuristic baselines under the same budget.

This negative row is the hinge between the gate result and the artifact-feedback result later in the paper. A plain neural proposer trained only on structural descriptors is not enough; the system becomes useful when verifier failures are promoted into reusable reasons. The generated C5 reason-learning row makes that repair measurable: at budget 8, `model_score` selects 7 unsafe rules and causes 172 validation false rejections, while reason-prior and trained-residual orderings recover useful certified rules with zero validation false rejections (Figure 8). Thus the paper does not claim "neural proposal wins"; it claims that verifier-grounded traces can turn failed proposals into the next proposal policy's training signal.

7. Axis 3: Verifier RL must preserve calibration

Starting from a single supervised checkpoint, we clone two RL branches: strict policy gradient on exact rewards, and the same gradient regularized with KL-to-supervised-prior + entropy.

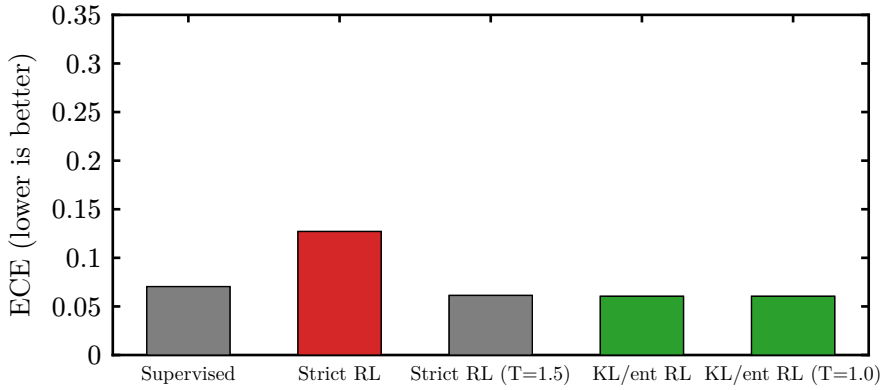


Figure 4: Expected calibration error by RL variant. Strict verifier RL degrades calibration (ECE 0.070 $\rightarrow$ 0.127) while leaving top-1 **unchanged at 9.13%** across all variants. Post-hoc temperature scaling needs $T = 1.5$ to recover; KL/entropy regularization fixes it at training time without that crutch.

Policy	**T**	**Top-1**	**Top-10**	**NLL**	**ECE**
Supervised	1.00	9.13%	78.01%	2.9170	0.0704
Strict verifier RL	1.00	9.13%	78.01%	3.0611	**0.1271**
Strict RL, temp-scaled	1.50	9.13%	78.01%	2.9063	0.0613
KL/entropy RL	1.00	9.13%	78.01%	2.9056	0.0604
KL/entropy RL, temp-scaled	1.00	9.13%	78.01%	2.9056	0.0604

Table 6: RL ablation, single cloned supervised checkpoint, default split.

The mechanism: strict policy gradient on a sparse exact reward signal pushes logit magnitudes without changing their relative ordering. The KL penalty bounds movement away from the reference policy; the entropy bonus discourages collapse to a single overconfident action. Together they reproduce the standard RLHF training stabilization, applied here to verifier rewards instead of preference rewards.

7.1. Reliability Diagram

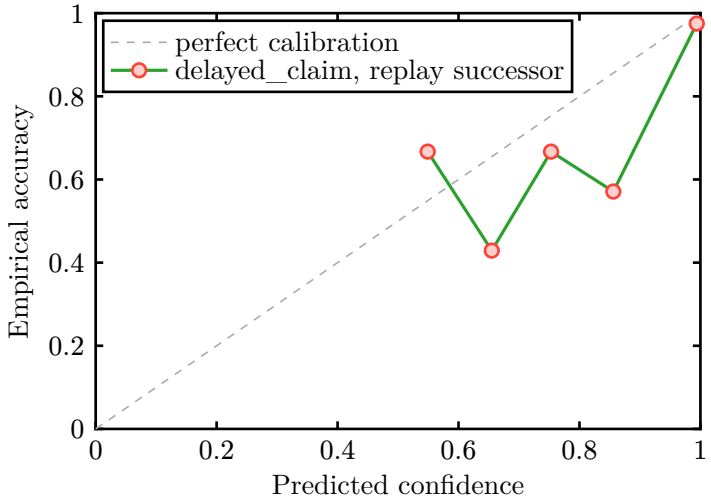


Figure 5: Reliability diagram for the trace-replay successor on `delayed_claim`. Bins from `results/latest/comparison.jsonl`. The high-confidence bin (894 of 1000 samples, $\hat{p} = 0.994$, $\hat{p}_{\text{emp}} = 0.975$) carries most of the mass; the low-population mid bins are noisier.

8. Cross-Task Learning

The same shared neural library, scorer architecture, and verifier-backed task contract instantiate across the task families in Table 3. This section reports how the learned component behaves on each. The reported numbers are produced by `make results` and live in `results/latest/comparison.jsonl`; the figures in this section are driven by `paper/data.typ`, autogenerated from those same artifacts by `paper/scripts/extract_learning_curves.py` so the figures cannot drift from the underlying runs.

8.1. Per-task gate ablation

Different tasks instantiate the certificate-gating loop differently: the prime-gap modular gate instance and grid-path have **gate ablations** in the strict sense (same scorer, with vs without the gate); delayed-claim has a **checkpoint succession** (supervised parent vs trace-replay successor); ARC-AGI has a **scorer-vs-gate** ablation where the two rows are different architectures (a neural cell scorer and a symbolic transform gate over the same task). A single common-axis bar chart would obscure those distinctions, so Figure 6 reports four small per-task panels with row labels that name what is being compared.

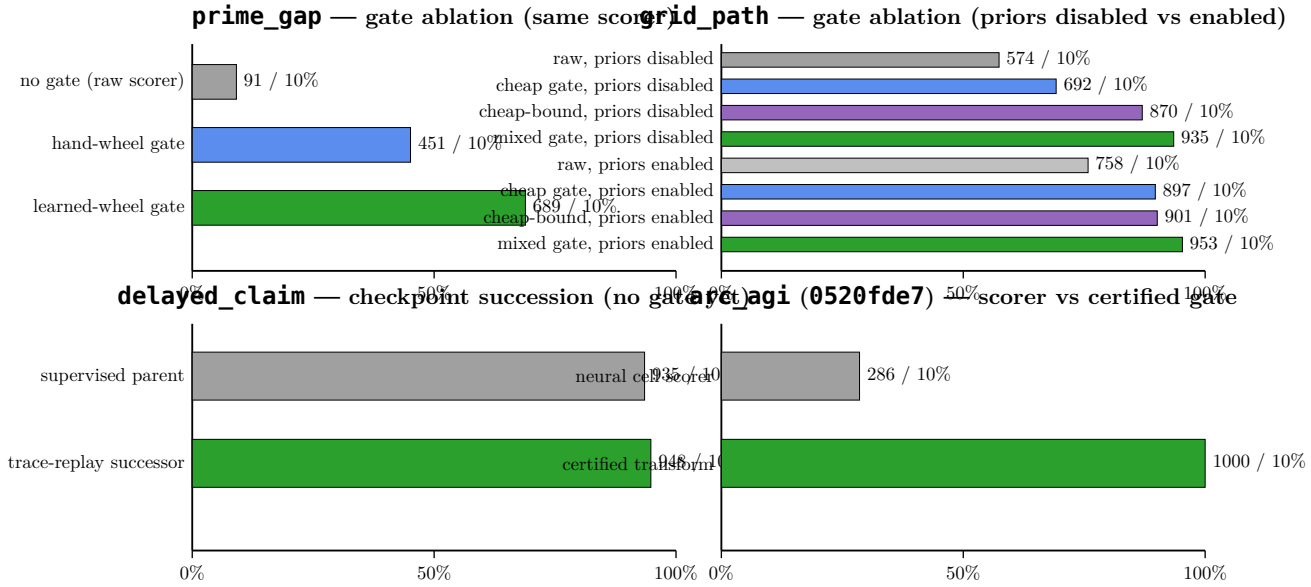


Figure 6: Per-task accuracy by ablation row. Each panel labels its own comparison so the rows are read against the right baseline: the prime-gap modular gate instance and grid-path are **gate ablations** (same scorer, with vs without the gate; grid-path additionally varies whether the Manhattan structural priors are enabled in the scorer features); delayed-claim is a **checkpoint succession** (supervised parent vs trace-replay successor on the same evaluation set, with parent-hash chain attesting identity); ARC-AGI is a **scorer-vs-gate** ablation where the two rows are different architectures over the same `0520fde7` task, not the same model with vs without a gate. The ARC `100.00%` is on nine held-out cells of one task; the public 400-task ARC training corpus has a 24/400 (≈6.0%) certified-gate floor under grammar v10, with the hypothesis-induction layer in Section 10 adding transform families v10 cannot synthesise (zero baseline-rate, 4/4 induction-rate on the synthetic suite under `crlp.arc_induction.parity.v1`).

8.2. Training curves per task

The four tasks expose learning on different timescales. `grid_path` and `delayed_claim` emit per-step training traces under `--trace-train`, so we plot rolling 50-step accuracy across a 4000-step supervised run. `prime_gap` does not emit a per-step training trace; its first-class binary reports per-epoch supervised loss on stdout, which decreases over 8 epochs and is what we plot. `arc_agi` trains its cell scorer in a single supervised block and exposes per-epoch eval only by re-running the binary with different `--`

`epochs` values, so we plot the top-1 / NLL curve over a 4-point sweep `{1, 2, 4, 8}`. Each panel labels its own x-axis honestly. The plotted data is imported verbatim from `paper/data.typ`, autogenerated by `paper/scripts/extract_learning_curves.py`.

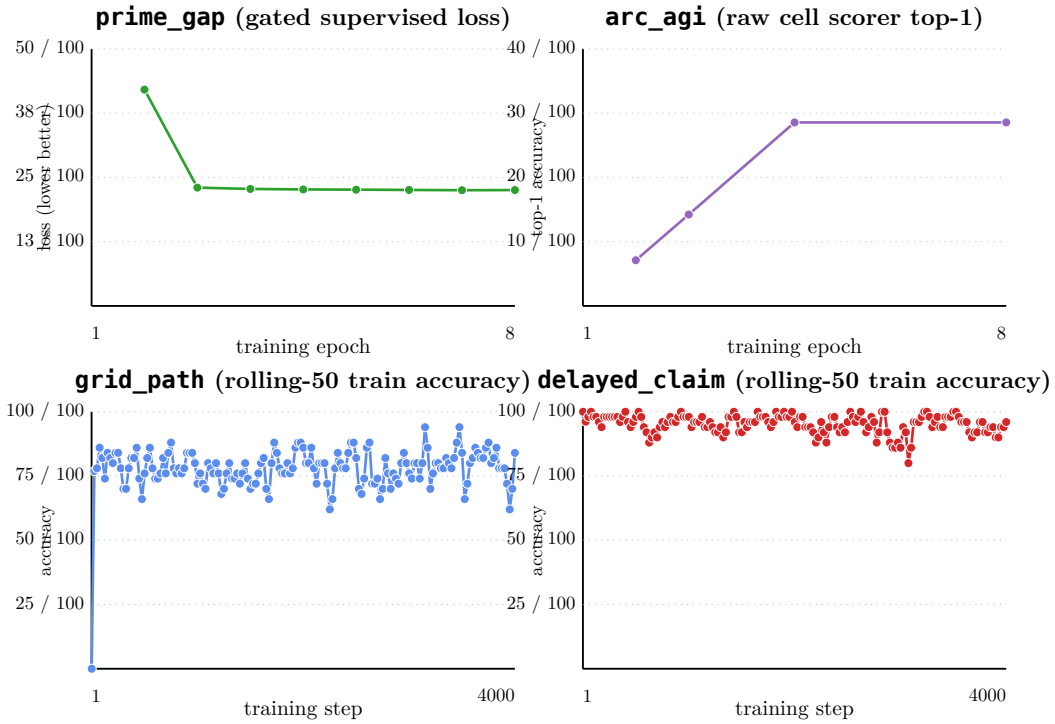


Figure 7: Per-task training-time curves on four different timescales, each labeled with its own y-axis: **`prime_gap`** plots the per-epoch supervised loss of the **gated** scorer (learned-wheel supervised), which decreases from 0.42 to 0.23 over 8 epochs and plateaus around epoch 3 — the raw scorer's top-1 is structurally flat at ≈12% on this minimal feature set because the 129-way candidate space exceeds the residual signal, which is the whole point of the gate. **`arc_agi`** plots per-epoch top-1 of the raw cell scorer on `0520fde7` across the sweep $\{1, 2, 4, 8\}$ epochs; the scorer climbs from 7.11% to 28.56% over the first four epochs and then plateaus, after which additional epochs only inflate NLL (visible as ECE drift in Figure 6). **`grid_path`** and **`delayed_claim`** plot rolling 50-step accuracy across the 4000-step supervised training trace produced under `--trace-train`. All four series are imported from `paper/data.typ`, autogenerated by `paper/scripts/extract_learning_curves.py` from the binaries' own outputs.

8.3. Per-task interpretation

Prime-gap modular gate instance. Figure 2 already shows the **9.13 → 45.12 → 68.87%** progression at final evaluation. The training curve in Figure 7 shows that the cell scorer alone reaches ≈20% top-1 — well below the hand-wheel and learned-wheel gated rows. The model is learning **something**, but on a 129-way action space the gate is the load-bearing component.

Grid-path. The model has access to Manhattan structural priors (`candidate_manhattan_to_goal`, `candidate_manhattan_delta`); the **prior-disabled** rows quantify their value. With priors enabled, the raw scorer is at 75.80%; the cheap-local gate lifts this to 89.70% (+13.9pp), the cheap-bound gate reaches 90.10% (+14.3pp), and the mixed topology+bound+exact-distance gate reaches 95.30% (+19.5pp). With priors disabled at evaluation time, the cheap-local gate lifts 57.40% -> 69.20%, the cheap-bound gate reaches 87.00%, and the mixed gate reaches 93.50%. The clean cross-domain gate claim is now the non-exact cheap-bound family; the distance-dominance rule remains an explicitly

labeled oracle-style upper bound. The family/budget proposer sweep is a robustness check: the default neural selector now targets train/validation residual marginal utility, which prevents overlapping clear-path rules from consuming the fixed budget, but it is not an "always wins" result at every tiny budget and seed.

Delayed-claim. The supervised parent reaches 93.5% accuracy and ECE 0.0363; the trace-replay successor improves to 94.8% / ECE 0.0166 over the same evaluation set with a parent-hash chain attesting identity. The production threshold+margin gate then certifies 4 rules for the neural, closed-loop, structural, and validation-oracle orderings, with zero held-out false rejections. The accepted threshold rules remove dominated `ignore`/`act_cautiously` actions; the accepted utility-margin rules resolve the hard `trust` versus `request_more_evidence` residual. On the held-out generator this leaves 1.00 candidate/action, so the gated row is 100.0% accurate with NLL 0.0000 and ECE 0.0000. The same direct gate stays exact on a shifted seed band, but clean-observation soundness is not the same thing as noisy-observation robustness: with observation noise 0.03 it records 23/1000 false rejections, and on a hard uncertainty+noise band it records 104/1000. The indirect risk-margin grammar is safer but less decisive: the structural/oracle row has zero false rejections under observation noise 0.03, but only 94.6% accuracy and 1.35 candidates. The reliability diagram in Figure 5 is from the replay successor before this deterministic gate.

ARC-AGI. On the fixed `0520fde7` task, the neural cell scorer reaches 28.56% top-1 with **ECE 0.5574 — substantially worse calibration than any of the other three tasks**. The cause is structural: the cell scorer must distribute probability mass over 10 candidate colors per output cell while seeing only three training pairs and 9 held-out cells, so the softmax has neither enough data nor enough structure to align confidence with accuracy. The certified transform gate then sidesteps the calibration question entirely: it produces a **unique** grammar-consistent prediction or no prediction, so the held-out top-1 is either 100% or the gate abstains. On the public ARC training corpus (400 tasks) the v10 grammar certifies 24/400 (≈6.0%) training and 4/400 (≈1.0%) evaluation tasks; the hypothesis-induction layer described in Section 10 adds **transform families v10 cannot synthesise**. Under the matched-baseline parity protocol `crlp.arc_induction.parity.v1`, on the synthetic move-to-anchor suite the baseline grammar certifies 0/4 tasks and the hypothesis-induction loader certifies 4/4 with zero regressions on the v10-supported tasks; the same harness applied to the public ARC corpus is reported below as a scoped evidence surface rather than as a claim of competitive ARC performance.

8.4. Cross-domain port

To show the substrate is not ARC-specific, the same `NNHypothesis` + `NNProgram` + evidence schema is instantiated in the `program_synthesis` task on a `filter_then_reduce` template (stages `enumerate → filter → reduce`, slots `filter.predicate_id`, `reduce.kind`, `output.type`). The existing certificate checker for the list-DSL abstractions accepts the compiled programs unchanged, and `make program-synthesis` emits **nine certified `crlp.hypothesis.evidence.v1` rows** (one per abstraction in the library: `count_positive`, `count_negative`, `count_even`, `sum_positive`, `sum_negative`, `any_positive`, `all_positive`, `first_positive_or_zero`, `any_even`). Eight of the nine cover one task-instance event each; `any_even` certifies but covers zero on the current fixture-task pool — the row appears in `results/latest/summary.md` as a "Program Synthesis Hypothesis Evidence" table. A corresponding `prime_gap` hypothesis cross-port is not yet part of the supported evidence.

8.5. One shared scorer across all four tasks

The cross-task panels in Figure 7 trained a separate `f_theta` for each task. As an ablation, we replace the four scorers with **one shared scorer of the same architecture** — a single `[96 → 64 → 32 → 1]` MLP that takes a task-tagged action row (a fixed 96-dimensional envelope with the task id as a one-hot prefix) and softmaxes over each task's candidate set. The shared scorer is trained on a uniform-

random draw across the four tasks at matched per-task update budget; for the same total budget we also train four single-task baselines. Numbers are means across three seeds: 16 epochs × 12 000 steps per seed, for 192 000 total updates, captured from `crlp.multitask.summary.v1` rows. **Three-seed steady state is what the paper claims; the single-seed snapshot from the earlier capture is retained in `paper/data.typ` as historical context only.**

Task	Cands	Shared top-1	Single top-1	Δ top-1 (3-seed)	Shared NLL	Single NLL
`prime_gap`	129	**14.1%**	13.6%	+0.4 pp	2.6648	2.6733
`grid_path`	4	90.9%	**91.1%**	−0.2 pp	0.3216	0.2999
`delayed_claim`	4	**98%**	97.9%	+0.1 pp	0.0472	0.0412
`arc_agi`	10	26.2%	**42.8%**	−16.6 pp	4.0827	8.2754

Table 7: Shared **one-MLP-on-all-four-tasks** vs **single-task** baseline at three-seed steady state, with 192 000 total updates per seed. Same `[96 → 64 → 32 → 1]` MLP, same task-tagged 96-feature envelope, same learning rate, three seeds, mean reported. At steady state the multi-seed picture is much quieter than the single-seed snapshot suggested: `delayed_claim` is statistically tied (+0.1pp); `prime_gap` and `grid_path` are tied within sub-1pp; **`arc_agi` keeps a 16.6pp single-task advantage** on the trained `0520fde7` task, down from the −28.6pp gap reported at the earlier 24 000-update budget. The "positive transfer on delayed_claim" framing from the one-seed snapshot did not survive replication and is retracted; the load-bearing multitask finding instead comes from the OOD evaluation in Table 8 below.

The load-bearing positive transfer signal is on OOD, not on the trained task. In the same-envelope OOD evaluation, we evaluate the same two ARC scorers (shared and single-task) — both trained only on `0520fde7` — on a held-out suite of 12 same-envelope synthetic ARC tasks (7×3 → 3×3 panel-composition tasks across families like `copy_left`, `copy_right`, `union`, `xor`, `row_any_intersection`). The shared scorer beats the single-task scorer on this OOD suite, even though it lost to it on the trained task in Table 7. This is consistent with the standard interpretation of single-task vs multi-task training: the single-task scorer is over-specialising to the 270 effective training rows of `0520fde7` (3 train pairs × 9 cells × 10 candidate colors) while the shared scorer is constrained by joint training on three unrelated tasks to find more transferable features.

Scorer	OOD top-1 (3-seed)	OOD NLL	OOD ECE
shared (multi-task)	**48.4%**	2.6278	0.4481
single-task ARC	42.8%	4.3723	0.482

Table 8: Same-envelope ARC out-of-distribution evaluation. Both scorers were trained only on `0520fde7`; this table evaluates each on a held-out suite of 12 synthetic 7×3 → 3×3 panel-composition tasks with the same 96-feature action envelope and the same 10-way color action space. Three seeds, aggregate row. **The shared multi-task scorer beats single-task by +5.57pp top-1 on OOD** despite losing by 16.6pp on the trained task. NLL drops by 40% under shared (2.63 vs 4.37), confirming that single-task is producing confidently wrong predictions on unseen tasks.

The reading is deliberately narrow. On the fixed ARC task, shared training loses in-distribution top-1; on same-envelope OOD, it improves top-1, NLL, and ECE. The certificate-gating contract makes that trade acceptable because the gate owns correctness where structure is already certified, while the scorer is most useful on the residual tasks the gate does not cover. Under a pure-scorer system this would be a direct memorization-vs-generalization tradeoff; under a verifier-gated system, the gate handles covered structure and the scorer can be optimized for residual ranking.

Two boundary checks keep the claim from becoming too broad. First, the calibration audit shows a real tradeoff: on fixed `0520fde7`, shared training lowers NLL (4.08 vs 8.28) while worsening ECE (0.6995 vs 0.5589); on same-envelope OOD, it improves both NLL (2.63 vs 4.37) and ECE (0.4481 vs 0.4820). Second, the delayed-claim leave-one-out row shows that transfer is sibling-specific rather than universal: all-domain training beats claim-only by +1.40pp, removing `grid_path` hurts most, and removing ARC slightly helps.

The capacity sweep rules out the simplest architectural explanation. Wider is not monotonically better: `[96→32→16→1]` is the closest shared ARC row to single-task (-1.23pp), while `[96→256→128→1]` only partially recovers the `64x32` gap. Delayed-claim stays positive or tied at every capacity. The remaining question is whether the current small nets are undertrained, saturated, or missing cumulative protocol state.

Hidden	Params	ARC shared	ARC single	ARC Δ	Claim Δ
`32x16`	3649	39.2%	40.4%	−1.23 pp	+0.43 pp
`64x32`	8321	26.2%	42.8%	−16.63 pp	+0.1 pp
`128x64`	20737	26.2%	40.4%	−14.27 pp	+0.13 pp
`256x128`	57857	34.5%	42.8%	−8.3 pp	+0.43 pp

Table 9: Fixed-task capacity sweep at the same three-seed, 192 000-update protocol. The ARC gap is **not** monotonic in trunk size: `32x16` is closest to single-task on fixed `0520fde7`, while `256x128` only partially recovers the `64x32` baseline. Delayed-claim remains positive or tied at every capacity.

The scorer-invariance audit fixes the boundary between learned scoring and certified ARC evidence. The public ARC transform gate and budgeted proposer path currently do not consume a multitask scorer checkpoint, so the comparison is an invariance audit over the scorer-independent path. Across 400 public train tasks and 400 public eval tasks, accepted rule sets, prediction hashes, loader summaries, and budgeted proposer aggregates were identical under the `shared_scorer_context` and `single_task_scorer_context` labels (`0` exact gate/loader mismatches and `0` proposer mismatches). This strengthens the verifier-correctness claim, but it also narrows the scorer claim: the audit does not show that the shared scorer improves public ARC search, only that the certified gate/proposer measurements are not contaminated by scorer choice in the current implementation.

8.6. Audited Evidence Surfaces

The experiment suite is tied to an explicit claim registry. Each row below is evaluated against a registered result artifact before the PDF is built, so unsupported or stale required claims fail before they become prose. The only non-blocking negative row remains the fixed-budget prime proposer claim, which is deliberately carried as a limitation. The compact matrix keeps the body readable, and the plotted claims below carry the short paper-facing explanations.

Claim	Family	Status	Required
C1.prime_gate_delta	C1	**supported**	yes
C2.prime_proposer_fixed_budget	C2	contradicted	no
C4.arc_ood_shared_vs_single	C4	**supported**	yes
C1.arc_gate_scorer_invariance	C1	**supported**	yes
C1.arc_public_evidence_surface	C1	**supported**	yes
C3.arc_ood_calibration_tradeoff	C3	**supported**	yes
C4.delayed_claim_sibling_transfer	C4	**supported**	yes
C4.multitask_capacity_tradeoff	C4	**supported**	yes
C5.rejected_candidate_reason_learning	C5	**supported**	yes
C5.hypothesis_library_promotion	C5	**supported**	yes
C8.trace_determinism_audit	C8	**supported**	yes
C9.signal_flight_replay_control	C9	**supported**	yes

Table 10: Generated claim status matrix. Required claims fail `make paper-support` if their registered evidence is missing, stale, underpowered, or contradicted.

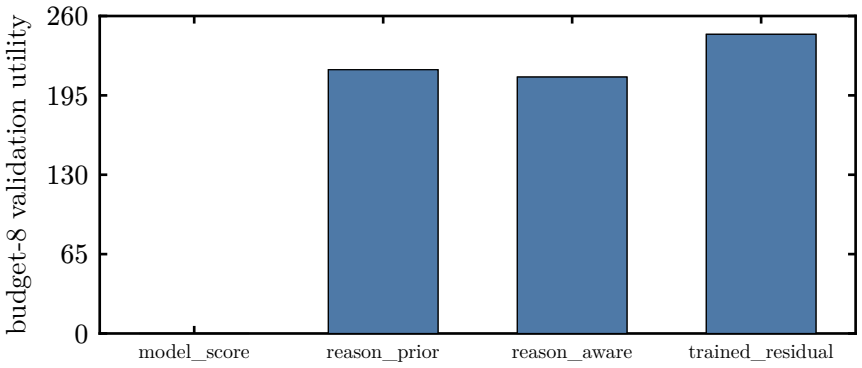


Figure 8: C5 rejected-candidate reason learning. reason learning repairs the prime proposer failure at budget 8: model_score selects 7 unsafe rows and causes 172 validation false rejections, while reason_prior reaches 6 accepted rules/216.0 utility and trained_residual reaches 7 accepted rules/245.0 utility with zero false rejections; prime/modular split sweeps pass, but delayed_claim has 8 mixed split rows.

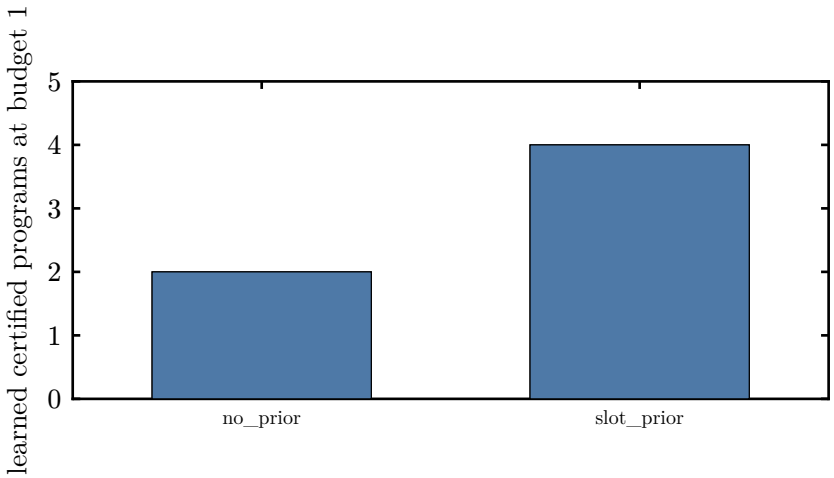


Figure 9: C5 certified hypothesis-library promotion. certified-library promotion supports fixture-level feedback: promoted move_to_anchor from 2 tasks/78 covered events with zero false rejections; slot priors raise learned certified@1 from 2 to 4 and utility@8 from 0.2119 to 0.5169.

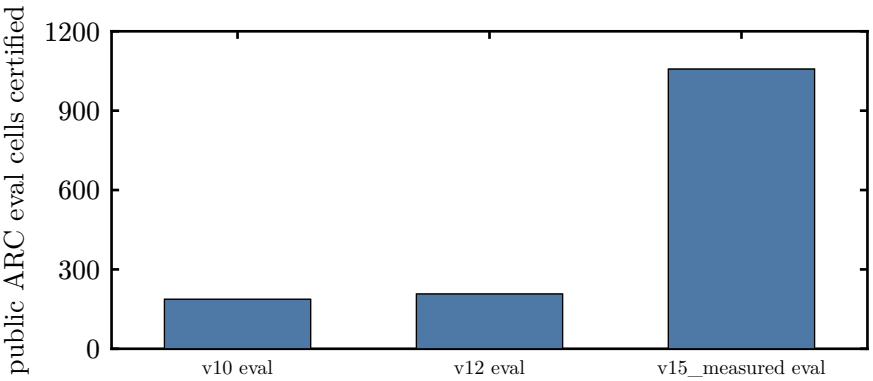


Figure 10: C1 public ARC evidence surface. public ARC evidence surface improves from v10 eval 4 tasks/187 cells to v15_measured eval 7 tasks/1057 cells; v15_measured train remains 24 tasks/1187 cells. The chart is intentionally scoped to certified covered cells, not overall ARC performance.

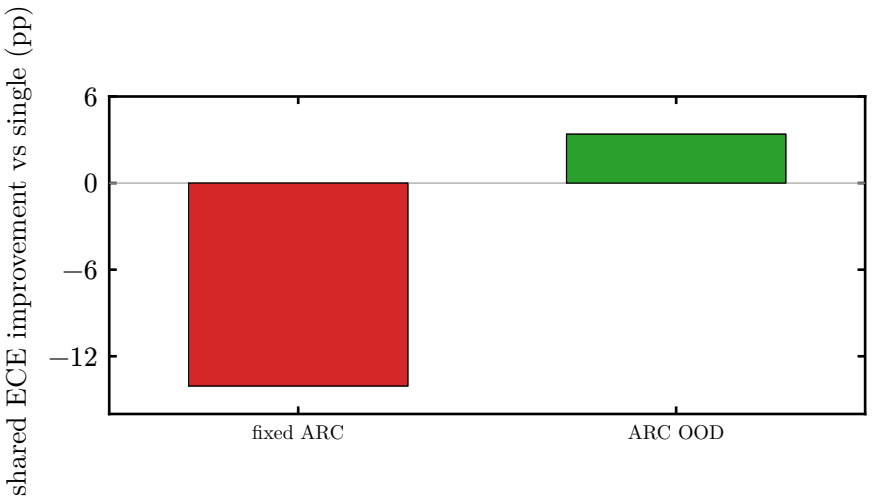


Figure 11: C3 ARC calibration tradeoff. Positive bars mean shared scoring has lower ECE than single-task scoring; negative bars mean worse ECE. ARC shared scorer is a calibration tradeoff: fixed-task NLL improves (4.0827 vs 8.2754) while ECE worsens (0.6995 vs 0.5589); OOD improves both NLL (2.6278 vs 4.3723) and ECE (0.4481 vs 0.4820).

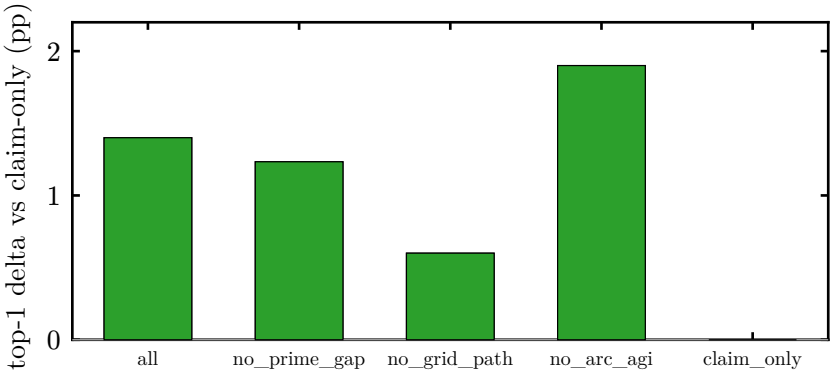


Figure 12: C4 delayed-claim sibling transfer. delayed_claim all-domain top-1 98.00% vs claim-only 96.60% (+1.40pp); removing grid_path falls to 97.20%, while removing ARC rises to 98.50%.

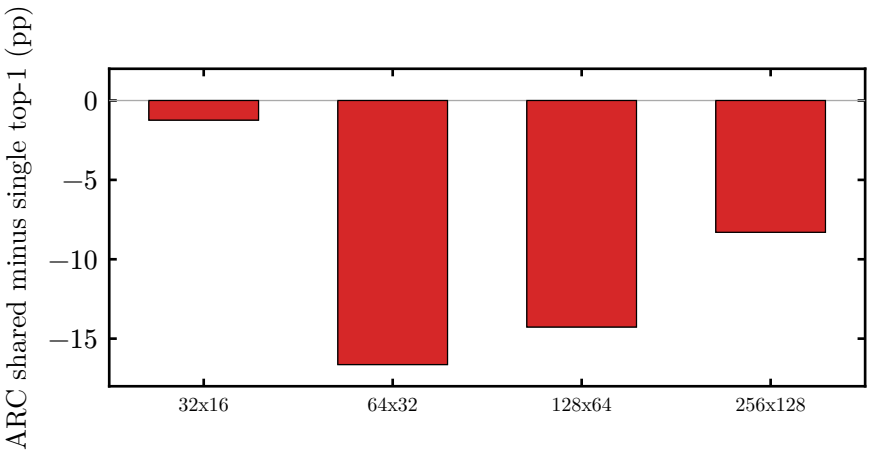


Figure 13: C4 capacity boundary. Negative bars mean the shared scorer still trails the single-task ARC scorer at that trunk size. capacity sweep rules out a simple wider-is-better ARC story: best fixed-ARC shared row is 32x16 at 39.20%, but every shared ARC row trails single-task; delayed_claim deltas remain +0.10pp to +0.43pp.

9. Application Vignette: Signal Flight

Signal is the practical reach test for the protocol, but the claim is intentionally narrow. The current result is not "a tiny net flies the game" and not "the live websocket pilot is neural-controlled." It is an offline counterfactual replay row: each sampled state enumerates nine WASD candidates, the Signal simulator replays the candidates, and the ranker learns from replay-certified utility, damage, and death outcomes.

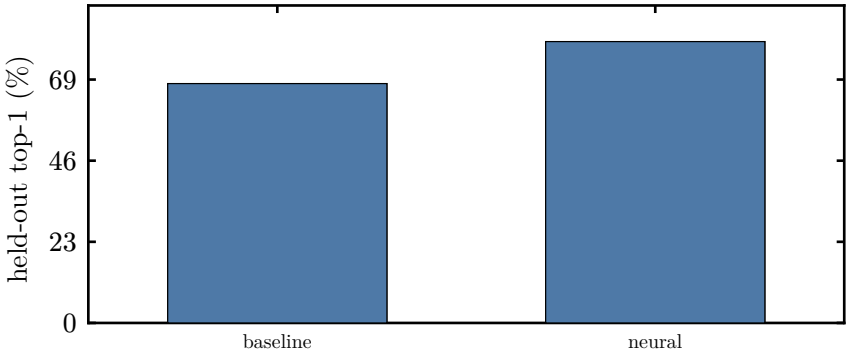


Figure 14: C9 Signal replay-control vignette. Signal offline replay ranker top-1 79.76% vs baseline 67.86% (+11.90pp), utility 0.1432 vs 0.1359, regret 0.0073 vs 0.0146, damage 2.38% vs 2.38%, deaths 0.00% vs 0.00%. The baseline is the primitive no-safety goal-heading WASD policy. This row supports bounded replay-certified ranking only; shadow inference and active neural control are outside the supported claim.

This is why Signal belongs in Table 3 even before it becomes a headline control experiment: it changes the environment class without changing the contract. Candidate enumeration is still explicit, the verifier is now authoritative simulator replay, the certificate is a replayed outcome record rather than a theorem-like proof, and the neural component still ranks candidates rather than owning safety. A live-control claim would require shadow inference first, then active control only after regret, damage, and death gates pass.

10. Artifact Feedback and Audit Contract

The hybrid architecture is the contribution; the audit contract is the discipline that makes the claims reproducible. Six pieces matter:

The generated C5 rows make the artifact-feedback loop explicit. Rejected candidate traces become reason priors and a trained residual that repair the prime/modular proposer economics without introducing false rejections (Figure 8). Repeated zero-false-rejection hypothesis bindings become a reusable library prior and feed that prior back into budgeted proposal ranking (Figure 9). In both cases, the method is the experiment: verified claim outcomes become the next round's proposal features, and the claim compiler decides whether the paper may say the loop helped.

- **Stable action IDs.** Trace rows carry `schema: "crlp.trace.v1"`, `trace_version: 1`, and `action_id_contract: "stable_unique_action_ids"`, plus a declared ranking policy and selected-score margin. Replay resolves `target_index` to the trace row's `action_id`, then maps that stable id back into the regenerated candidate row. The trace determinism audit projects rows onto replay-semantic fields, hashes that projection, and reports the count of `rank_fp_sensitive` decisions rather than requiring bit-identical logits.
- **Checkpoint identity.** SHA-256 over the exact checkpoint bytes, plus `parent_model_hash`, `feature_encoder_version`, and `feature_set` in metadata. Successor models declare their parent; comparison reports carry both hashes and the evaluated `feature_set`.
- **Hypothesis layer.** `NNHypothesis` (header `include/nn_hypothesis.h`) sits one level above `NNProgram`: a **template** declares a typed operator skeleton as a sequence of stages with named, typed slots; a **binding** fills each slot with a concrete value inferred from observed task evidence; `nn_hypothesis_compile` produces a composed `NNProgram` whose canonical id binds back to the hypothesis template, so the existing verifier and closed-loop policy treat it as any other program. `crlp.hypothesis.evidence.v1` rows are emitted by `nn_hypothesis_write_evidence_json`, a library function reused by both `arc_agi` and `program_synthesis`. The provenance contract holds: state-observation slot values are `NN_FEATURE_STATE_OBSERVATION`, gate-derived support / residual gain appear only on the evidence row, never on the hypothesis feature vector that a selection-mode proposer scores against.
- **Certificate-gated baselines.** `crlp.arc_induction.parity.v1` records the matched baseline-vs-induction certification delta per split (training, evaluation) on the public ARC corpus or any synthetic split; `crlp.program_proposer.summary.v1` records matched-budget rule-set comparisons across `{random, grammar-order, structural, learned, closed-loop, oracle}`. Both schemas attest **what was held fixed** between rows, which is the load-bearing thing for cross-task comparison.
- **Pure C, no third-party deps.** The neural library is dense layers, sigmoid/ReLU/tanh/linear, MSE and softmax cross-entropy, SGD, batched 4-by-4 tiled forward/backward, cloned networks, action scoring, pairwise ranking loss, and cached policy-gradient updates with optional KL/entropy regularization. ≈3k lines for the core; the current certified modular gate implementation is split into focused C modules for data, features, rule search, training, evaluation, and replay.
- **CI-baseline runs.** A pinned `ubuntu-24.04` + `gcc-13` image runs `make check` and the ARC induction smoke on every push and PR, runs `make results` nightly, captures `crlp.environment.v1` for runner identity, and emits a **timing-stripped** `comparison.canonical.jsonl` so the diff that gates "should the paper rewrite?" cannot be triggered by wallclock noise. Laptop runs produce the same artifacts; drift shows up as a visible `cc_version` or `git_dirty` difference rather than as silent numeric drift.

11. Limitations

L1. **Proposer learning remains weak (Axis 2).** The fixed-budget baseline shows grammar order and a simple structural heuristic beat the current neural proposer on both the unconditional candidate grammar and the conditional gap grammar. The load-bearing claim is certificate-gated rule discovery, not learned proposer superiority. The negative result is reported deliberately.

L2. **Cross-task verifier-RL comparison missing (Axis 3).** Calibration evidence is on the prime-gap modular gate instance only. The pipeline supports running strict-vs-KL on grid-path and delayed-claim; the comparison is not in canonical results yet.

L3. **Limited cross-domain gate coverage (Axis 1).** The prime-gap modular gate instance and grid-path have the strongest **gate ablations** in the strict sense. Delayed-claim now has a production threshold+margin gate that resolves its held-out deterministic generator, including the hard `trust` versus `request_more_evidence` residual, and it now reports shifted/noisy stress rows. Those rows are mixed: the direct grammar is exact under clean seed shift but false-rejects under observation noise, while the indirect proxy avoids false rejections but leaves a larger residual. This is evidence of the robustness frontier, not a broad noisy-claim solution. ARC-AGI's "gate" is a different architecture (transform grammar) over the same task, not the same scorer with vs without filtering, so it is reported separately in Figure 6.

L4. **Statistical rigor.** Default `make prime-gap-experiments` runs 1 seed; the harness supports more (`PRIME_GAP_EXPERIMENT_SEEDS=…`). The $9.13 \to 45.12 \to 68.87\%$ deltas, the 73.22% baseline saving, the grid-path 75.80 -> 90.10 cheap-bound gate delta, the 95.30 mixed-gate upper bound, and the modular-rule-search negative result all need confidence intervals across ≥ 5 seeds before any of them should be treated as a stable property.

L5. **Learned-wheel calibration tradeoff** needs a reliability-diagram pass. The learned wheel improves ranking sharply (Table 4) but has higher ECE (0.1402) than the raw minimal row (0.1076).

L6. **ARC ECE outlier.** The 0.5574 ECE on the `0520fde7` cell scorer is reported, with the structural explanation (10 colors $\times$ 3 train pairs is too sparse for the softmax to align with accuracy). It is not yet replicated across multiple ARC tasks; the structural argument is suggestive, not proof.

L7. **Signal is offline replay, not live neural control.** The Signal row is a bounded counterfactual WASD replay vignette. It supports the claim that replay-certified control candidates can train a tiny ranker against a primitive baseline, but the live websocket pilot remains outside the supported claim until shadow inference and active-control safety gates are measured.

L8. **No competitive large-system baseline.** The related-work comparison now positions the protocol against learned verifiers, RLVR, AlphaGeometry-style formal search, DreamCoder, Hypothesis Search, and ARC systems, but the artifact does not empirically compare against those large systems. That is acceptable for the protocol claim; it would not be acceptable for a claim of competitive theorem proving, language-model reasoning, or ARC performance.

L9. **Single-runner numbers.** All headline percentages were captured from a single CI baseline run (`ubuntu-24.04` + `gcc-13`, see `results/latest/environment.json`). Laptop runs produce different `model_ms_per_sample` and minor numerical differences in `nll` / `ece` at the third decimal; the canonical pipeline strips timing fields and now exposes a replay-semantic trace hash plus `rank_fp_sensitive` counts, but a cross-machine double-blind has not yet been run. Raw logits remain diagnostics rather than semantic trace identity.

Claim	Status	Generated limitation candidate
C2.prime_proposer_fixed_budget	contradicted	static neural proposer trails deterministic baselines on at least one grammar; candidate static=72.49 saved vs grammar=73.22, conditional static=2.15 saved vs grammar=55.57.

Table 11: Generated limitation candidates from the paper-claim compiler. The required claim support gate is currently passing. Optional contradicted claims remain visible here instead of blocking PDF compilation.

12. Conclusion

The architectural claim is **certificate-gating**: when an exact verifier is available, the scorer should consume the survivors of a soundness-proven symbolic filter, and that filter itself can be synthesised by a rule proposer gated on per-rule certificates rather than on empirical support. The constructive claim is **verifier-grounded rule and hypothesis synthesis**: the same `NNHypothesis` + `NNProgram` substrate certifies transforms on ARC that the hand-coded grammar cannot synthesise, and the same substrate emits certified rows in `program_synthesis` without modification — domain neutrality is exercised, not asserted. The training-stability claim is **KL-regularized verifier RL**: strict policy gradient on sparse exact rewards inflates logit scale on the prime-gap modular gate instance while leaving top-1 fixed; a standard KL-to-prior + entropy term restores calibration at unchanged ranking. The application claim is **replay-grounded control**: Signal shows the same bounded ranking contract on real-time WASD decisions, while keeping simulator replay and safety outcomes outside the model. These claims require the protocol to be falsifiable, audit-clean, and cross-task. The negative result on Axis 2 (grammar order beats the current neural proposer at fixed budget) is the cleanest evidence so far that the protocol is in fact falsifiable. The broader implication is pragmatic: when correctness, safety, or auditability matter, end-to-end learning should not be asked to own the whole decision. Verifier-grounded ranking substrates provide an auditable bridge: symbolic checks own admissibility, neural components spend capacity on search and prioritization, and the trace artifacts make both success and failure reusable.

Bibliography

[1] DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," *arXiv preprint arXiv:2501.12948*, 2025.

[2] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[3] N. Stiennon *et al.*, "Learning to summarize from human feedback," *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[4] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[5] K. Cobbe *et al.*, "Training Verifiers to Solve Math Word Problems," *arXiv preprint arXiv:2110.14168*, 2021.

[6] H. Lightman *et al.*, "Let's Verify Step by Step," *arXiv preprint arXiv:2305.20050*, 2023.

[7] J. Uesato *et al.*, "Solving math word problems with process- and outcome-based feedback," *arXiv preprint arXiv:2211.14275*, 2022.

[8] A. Hosseini, X. Yuan, N. Malkin, A. Courville, A. Sordoni, and R. Agarwal, "V-STaR: Training Verifiers for Self-Taught Reasoners," *arXiv preprint arXiv:2402.06457*, 2024.

[9] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[10] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust Region Policy Optimization," in *International Conference on Machine Learning (ICML)*, 2015.

[11] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," in *International Conference on Machine Learning (ICML)*, 2017.

[12] T. H. Trinh, Y. Wu, Q. V. Le, H. He, and T. Luong, "Solving olympiad geometry without human demonstrations," *Nature*, vol. 625, pp. 476–482, 2024.

[13] Y. Chervonyi *et al.*, "Gold-medalist Performance in Solving Olympiad Geometry with AlphaGeometry2," *arXiv preprint arXiv:2502.03544*, 2025.

[14] S. Polu and I. Sutskever, "Generative Language Modeling for Automated Theorem Proving," *arXiv preprint arXiv:2009.03393*, 2020.

[15] K. Ellis *et al.*, "DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning," *arXiv preprint arXiv:2006.08381*, 2021.

[16] R. Wang, E. Zelikman, G. Poesia, Y. Pu, N. Haber, and N. D. Goodman, "Hypothesis Search: Inductive Reasoning with Language Models," in *International Conference on Learning Representations (ICLR)*, 2024.

[17] F. Chollet, "On the Measure of Intelligence," *arXiv preprint arXiv:1911.01547*, 2019.

[18] F. Chollet, M. Knoop, G. Kamradt, and B. Landers, "ARC Prize 2024: Technical Report," *arXiv preprint arXiv:2412.04604*, 2024.

[19] E. Akyürek, M. Damani, L. Qiu, H. Guo, Y. Kim, and J. Andreas, "The Surprising Effectiveness of Test-Time Training for Abstract Reasoning," *arXiv preprint arXiv:2411.07279*, 2024.